

第5章 Spark | 开卷速查版

本章一句话：**Spark** 是为了解决 **MapReduce 磁盘 I/O 多、调度开销大、迭代计算慢、易用性差** 而出现的分布式计算框架。它的核心是 **RDD**，用 **内存计算 + DAG 执行 + lineage 容错** 提高批处理、迭代计算和交互式计算效率。🌟

常考题型： Spark vs MapReduce；RDD 是什么；Transformation / Action 判断；Spark WordCount；惰性执行；DAG、Stage、Task 怎么形成；窄依赖/宽依赖判断；join 是窄还是宽；RDD 如何容错。

翻页关键词： 动机 → 特点 → RDD → Transformation/Action → WordCount → Lazy Execution → 架构/运行模式 → DAG → Stage/Task → RDD组成 → 依赖 → 容错。

1. Spark 为什么出现：动机一，MapReduce 性能问题

MapReduce 的问题主要不是“不能算”，而是**算得不够快、不够灵活**。

磁盘 I/O 多： Map 中间结果写本地磁盘，Reduce 输出写 HDFS；多个 MR 作业串起来时，前一个作业输出到 HDFS，后一个再从 HDFS 读，反复落盘。

调度开销大： Map / Reduce Task 在传统 MR 中常以进程方式运行，启动和调度开销较大。

没有充分利用内存： MR 每一轮都偏向磁盘读写，不能很好缓存中间结果。

迭代计算慢： K-means、PageRank 这类算法需要反复使用同一批数据，MR 每轮都读写 HDFS，产生大量冗余 I/O。

易用性不强： MR 需要写较多 Java 代码，表达复杂计算不够方便。

PPT里把 MapReduce 性能问题总结为：磁盘 I/O 多、调度开销大、没有充分利用内存计算、迭代计算有大量冗余 I/O。

考试写法： Spark 出现的主要原因是 MapReduce 在复杂计算和迭代计算中效率较低。MapReduce 中间结果频繁写磁盘，多轮作业之间通过 HDFS 交换数据，任务启动和调度开销大，也不能充分利用内存缓存，因此在 K-means、PageRank 等迭代场景下会产生大量冗余 I/O。

2. Spark 为什么出现：动机二，框架多样化问题

早期大数据系统里，不同任务往往用不同框架：

```
批处理：MapReduce / Hive / Pig
流处理：Storm
交互式查询：Impala
```

这样会带来三个问题：

数据重复和搬运成本高： 同一份数据可能在多个系统间复制、转换、搬运。

系统复杂度高： 多套框架都要维护，运维困难。

无法跨范式计算： 同一个计算逻辑，批处理、流处理、交互式查询可能要写多个版本。

Spark 的目标之一是：用一个统一框架支持 **批处理、流式计算、交互式计算**。PPT也明确说 Spark 统一支持多种计算模式。

考试写法： Spark 试图统一批处理、流式计算和交互式计算，减少多框架并存带来的数据重复、数据搬运、系统运维复杂和同一计算逻辑重复实现的问题。

3. Spark 的特点

Spark 特点可以背四个：**高效、易用、统一、可集成**。

高效： Spark 通常比 MR 快，核心原因不是“魔法”，而是 **内存计算、Cache、DAG 引擎、多线程 Task**。Cache 可以减少重复读数据；DAG 可以减少多个计算阶段之间写 HDFS；线程级 Task 启动开销比进程小。PPT中把 Spark 高效的原因写为内存计算引擎、Cache 机制、DAG 引擎、多线程池模型。

易用： Spark 提供类似 SQL 的高级算子，支持 Java、Scala、Python、R，代码量通常比 MR 少。

统一： Spark 支持批处理、流式计算、交互式计算。

可集成： Spark 没有自己的存储和资源管理系统，可以读写 HDFS / HBase，并与 YARN 集成。

考试写法： Spark 的特点包括高效、易用、统一和易集成。它通过内存计算、RDD Cache、DAG 执行和线程级 Task 减少磁盘 I/O 和调度开销；同时提供高级 API，支持多种语言，并能与 HDFS、HBase、YARN 等 Hadoop 生态系统集成。

4. Spark vs MapReduce：本质差别

Spark 和 MR 在表达上都能做批处理，但**执行方式不同**。

对比点	MapReduce	Spark
核心模型	Map + Reduce 两阶段	RDD + DAG
中间结果	多轮作业常写 HDFS	尽量在内存/RDD中传递
迭代计算	每轮读写 HDFS	Cache 复用数据
执行计划	Map 后 Reduce，模型较固定	DAG 可包含多个算子
Task	进程开销相对大	Executor 内线程级 Task
容错	重跑失败 Task	lineage 重算丢失 RDD partition

最核心一句： MapReduce 的中间结果经常要落到 HDFS；Spark 通过 RDD 和 DAG 让多个算子链式执行，中间结果不一定落 HDFS。

考试写法： Spark 相比 MapReduce 的本质差别在于，Spark 以 RDD 和 DAG 组织计算，可以把多个转换操作形成一个执行图，并通过内存缓存和 lineage 容错减少重复磁盘 I/O；而 MapReduce 以 Map 和 Reduce 两阶段为主，多轮作业之间通常通过 HDFS 交换中间结果。

5. Spark 核心：RDD

RDD = Resilient Distributed Dataset，弹性分布式数据集。

可以把 RDD 理解成：分布在多台机器上的只读大数组。

PPT里说，每个 RDD 是一份“分布在多台机器上的数据”，支持并行计算，可溯源，可以根据计算过程重新生成，是 Spark 自动容错的核心；RDD 可存内存也可存磁盘，能避免重复 I/O。

RDD 的关键词：

Resilient，弹性：丢失后能根据 lineage 重新计算。**Distributed**，分布式：数据分布在多台机器上。

Dataset，数据集：逻辑上是一组对象集合。**只读**：RDD 创建后不能原地修改，每次 Transformation 都产生新的 RDD。**分区**：一个 RDD 由多个 **Partition** 组成。**可缓存**：可以放内存或磁盘中复用。**可溯源**：记录自己由哪些父 RDD 经过哪些算子计算而来。

考试写法：RDD 是 Spark 的核心抽象，即弹性分布式数据集。它是分布在集群中的只读对象集合，由多个 Partition 构成，可以存储在内存或磁盘中，并通过 Transformation 从已有数据或已有 RDD 构造。RDD 记录 lineage，因此某个分区丢失后可以根据血统关系重新计算。

6. RDD 和 HDFS 文件的本质差别

这个点你标了“本质差别是什么”，要直接背。

HDFS 文件：是磁盘上的文件，只知道数据存在那里，不知道这些数据是怎么计算出来的。它**不可溯源**。

RDD：是 Spark 的计算抽象，不只是数据，还记录“由哪些父 RDD、经过哪些算子得到”。它**可溯源**，所以能自动容错。

PPT里明确对比：HDFS 文件不可溯源，是磁盘文件，和读入内存后的数据结构缺乏联系；RDD 可溯源，是自动容错核心。

考试写法：HDFS 文件只是底层磁盘文件，不记录数据的计算来源；RDD 不仅表示分布式数据集，还记录由哪些父 RDD 经过哪些转换算子生成，即 lineage。由于 RDD 可溯源，Spark 可以在分区丢失时重新计算，而不必像 HDFS 一样依赖多副本保存中间结果。

7. RDD 特点二：可以存储在内存或磁盘中

Spark 允许把 RDD 缓存在 **内存** 或 **磁盘** 中，方便后续复用。比如 K-means 中训练数据要反复使用，缓存后就不用每轮从 HDFS 重新读。

Cache 的意义：减少重复 I/O，提升迭代计算和交互式查询性能。

考试写法：RDD 可以缓存到内存或磁盘中，以便后续计算重复使用。对于 K-means、PageRank 等迭代算法，缓存 RDD 可以避免每轮都从 HDFS 重新读取数据，从而显著减少磁盘 I/O。

8. RDD 特点三：通过 Transformation 构造

RDD 不会被原地修改。每次对 RDD 做转换，都会产生一个新的 RDD。

RDD 来源有两类：

1. 从外部数据读入：如 HDFS 文件、Scala 集合。
2. 从已有 RDD 经过 Transformation 转换产生。

常见 Transformation：

```
map
filter
flatMap
groupByKey
reduceByKey
join
```

PPT里强调：Transformation 输入是 RDD，输出也是 RDD；RDD 可以从 Scala 集合或 HDFS 数据集读入，也可以由已有 RDD 转换产生。

9. Transformation 和 Action 判断

这个可能考判断题或简答。

Transformation：输入 RDD，输出还是 RDD，只记录转换关系，不立刻执行。

例如：

```
map
flatMap
filter
reduceByKey
groupByKey
join
union
distinct
```

Action：输入 RDD，输出不再是 RDD，或者把结果写到外部系统，会触发真正执行。

例如：

```
count
collect
reduce
take
saveAsTextFile
foreach
```

判断口诀：

结果还是 RDD → Transformation
结果变成本地值 / 写出去了 → Action

PPT中定义 Transformation 输入输出都是 RDD，Action 输入是 RDD、输出不再是 RDD，并列出了 count、reduce、saveAsTextFile 等例子。

考试写法：Transformation 是从一个 RDD 生成另一个 RDD 的操作，如 map、filter、reduceByKey；Action 是触发实际计算并返回结果或写出结果的操作，如 count、collect、saveAsTextFile。判断时看输出是否仍为 RDD：如果仍是 RDD，就是 Transformation；如果输出值或写外部系统，就是 Action。

10. Spark 版 WordCount

Spark 版 WordCount 标准写法：

```
val text = sc.textFile("hdfs://input")
val words = text.flatMap(line => line.split(" "))
val pairs = words.map(word => (word, 1))
val counts = pairs.reduceByKey((a, b) => a + b)
counts.saveAsTextFile("hdfs://output")
```

流程：

```
textFile
-> flatMap: 一行拆成多个 word
-> map: word 变成 (word, 1)
-> reduceByKey: 按 word 聚合求和
-> saveAsTextFile: 写出结果
```

注意：

flatMap、map、reduceByKey 是 Transformation。 它们只记录计算逻辑，不真正执行。

saveAsTextFile 是 Action。 它触发整个 DAG 真正执行。

PPT中明确写 Spark 版 WordCount：flatMap、map、reduceByKey 都是 Transformation，记录计算逻辑；saveAsTextFile 是 Action。

考试写法：Spark WordCount 先通过 textFile 读取文本形成 RDD，再用 flatMap 将每行拆成单词，用 map 转成 `<word, 1>`，用 reduceByKey 按 word 聚合计数，最后用 saveAsTextFile 写出结果。其中 flatMap、map、reduceByKey 是 Transformation，saveAsTextFile 是 Action，并触发实际执行。

11. 惰性执行 Lazy Execution

惰性执行 是 Spark 的重要特点。

意思是：Transformation 不会马上执行，只会记录 RDD 之间的依赖关系；直到遇到 Action，Spark 才根据整个 RDD 依赖图生成 DAG 并执行。

比如：

```
val a = sc.textFile("input")
val b = a.flatMap(...)
val c = b.map(...)
val d = c.reduceByKey(...)
```

到这里都不会真正跑。只有：

```
d.saveAsTextFile("output")
```

才会触发执行。

为什么要惰性执行？ 因为 Spark 可以等到知道完整计算链后，再统一优化和划分 Stage，而不是每来一个算子就马上执行。

PPT里也写：Transformation 只会记录 RDD 转换关系，并不会触发计算；Action 是触发分布式执行的算子。

考试写法： Spark 采用惰性执行机制。Transformation 只记录 RDD 之间的转换关系，不立即执行；当遇到 Action 时，Spark 才根据完整 lineage 构建 DAG，划分 Stage 和 Task，并提交到集群执行。惰性执行有利于全局优化和减少不必要计算。

12. Spark 架构

Spark 应用通常由一个 **Driver** 和多个 **Executor** 组成。

Driver： 主控程序。负责创建 SparkContext、构建 DAG、划分 Stage、分发 Task、监控执行状态、失败重试。

Executor： 计算进程。运行在工作节点上，负责执行 Task，管理内存缓存，并和 Driver 通信。

Task： 最小执行单元。一个 Task 通常处理一个 Partition，在 Executor 内以线程方式运行。

Container： YARN 分配的资源单位，用来启动 Executor 等进程。

PPT中对 Spark 架构的定义是：Driver 分发 Task、监控状态、失败重试；Executor 执行 Task、管理内存和 Cache 数据；Task 是最小执行单元，一个 Task 处理一个 Partition。

考试写法： Spark 应用由 Driver 和多个 Executor 组成。Driver 负责生成执行计划、划分和调度 Task，并监控执行状态；Executor 负责运行 Task、管理缓存数据并与 Driver 通信。Task 是 Spark 的最小执行单元，通常一个 Task 处理一个 RDD Partition。

13. Spark 运行模式

Spark 没有自己的存储和资源管理，通常依赖 HDFS 和 YARN 等系统。运行模式主要有：

Local： 本地模式，单机多线程运行，用于测试和调试。`Local[K]` 表示启动 K 个线程，`Local[*]` 表示使用和 CPU 个数相近的线程数。

Standalone： Spark 自带的独立集群模式。

YARN / Mesos： 运行在资源管理系统上。公司里常用 YARN。

Yarn-client： Driver 在客户端本地运行，方便调试和看日志，但客户端挂了会影响任务。

Yarn-cluster： Driver 在集群中运行，更适合生产环境。

PPT中列出运行模式：Local、Standalone、Yarn/Mesos，并说明 Spark on Yarn 有 Yarn-client 和 Yarn-cluster 两种模式。

考试写法： Spark 支持 Local、Standalone、YARN 和 Mesos 等运行模式。Local 用于本地测试，Standalone 是 Spark 自带集群模式，YARN/Mesos 依赖外部资源管理系统。在 Spark on YARN 中，client 模式下 Driver 运行在客户端，便于调试；cluster 模式下 Driver 运行在集群中，更适合生产运行。

14. DAG 执行：Spark 怎么从代码变成任务

Spark 程序不会像 MapReduce 那样只拆成 Map 和 Reduce 两步，而是把 RDD 转换关系组织成一个 **DAG**，有向无环图。

执行流程：

1. 用户写 RDD 程序。
2. Transformation 生成逻辑 DAG。
3. 遇到 Action，提交一个 Job。
4. Spark 根据依赖关系划分 Stage。
5. 每个 Stage 按 Partition 生成多个 Task。
6. Driver 调度 Task。
7. Executor 执行 Task。

PPT里写 Spark 引擎内部执行流程：通过 RDD 创建逻辑计划 DAG，为 DAG 生成物理查询计划，调度 Task，分布式执行 Task。

考试写法： Spark 首先根据 RDD 的 Transformation 构建逻辑 DAG；遇到 Action 后，将 DAG 转换为物理执行计划，按照依赖关系划分 Stage，再将每个 Stage 划分为多个 Task。Driver 负责调度 Task，Executor 负责分布式执行。

15. Application、Job、Stage、Task 的关系

这个层级很容易考：

```
Application
-> Job
  -> Stage
    -> Task
```

Application: 一个 Spark 应用，对应一个 SparkContext。

Job: 每个 Action 会触发一个 Job。也可以说，一个 Action 以及它之前依赖的 Transformation 序列构成一个 Job。

Stage: 一个 Job 会被切成多个 Stage。Stage 的边界通常由 **宽依赖 / Shuffle** 决定。

Task: 一个 Stage 由多个并发 Task 组成。每个 Task 通常处理一个 Partition。

PPT总结为：每个 Application 包含若干 Job；每个 Action 以及之前的 Transformation 序列构成一个 Job；每个 Job 分成若干 Stage；每个 Stage 包含若干并发执行的 Task。

考试写法: Spark 的执行层次是 Application、Job、Stage、Task。一个 SparkContext 对应一个 Application；每个 Action 触发一个 Job；一个 Job 根据依赖关系被切分为多个 Stage；每个 Stage 按数据分区生成多个 Task 并行执行。

16. Stage 是怎么切的？

关键口诀: 看到 **Shuffle / 宽依赖**，就切 **Stage**。

Spark 会尽量把连续的窄依赖操作放在同一个 Stage 里流水线执行；遇到需要 Shuffle 的宽依赖操作，就切开 Stage。

例子：

```
textFile -> flatMap -> map -> reduceByKey -> saveAsTextFile
```

其中：

```
textFile -> flatMap -> map
```

都是窄依赖，可以放在前一个 Stage。

reduceByKey 需要按 key 聚合，通常会产生 Shuffle，所以在这里切 Stage。

可以写成：

```
Stage 1: textFile + flatMap + map + shuffle write
Stage 2: shuffle read + reduceByKey + saveAsTextFile
```


考试写法： Spark 根据依赖关系划分 Stage。窄依赖的多个 Transformation 可以放在同一个 Stage 中流水线执行；遇到宽依赖或 Shuffle 操作时，需要等待父 Stage 的 Shuffle 输出，因此会在该处切分 Stage。

17. Task 是怎么形成的？

Task 数量和 Partition 有关。

一个 Stage 中通常有多少个 Partition，就会生成多少个 Task。PPT里说：First stage 的 Task 数目由 HDFS block 或 HBase region 个数决定；其他 Stage 的 Task 数目由用户设置，默认与第一个阶段相等。

记法：

```
一个 Stage 的一个 Partition -> 一个 Task
```

对于第一阶段：

```
HDFS 输入有几个 block  
-> 通常形成几个 input partition  
-> 对应几个 Task
```

对于 Shuffle 后阶段：

```
由 shuffle partition 数决定  
通常可由用户参数设置
```

考试写法： Spark 中每个 Stage 会按照 Partition 划分为多个 Task。通常一个 Partition 对应一个 Task。第一阶段的 Task 数目一般由输入数据的 HDFS block 或 HBase region 数量决定；后续 Stage 的 Task 数目通常由 Shuffle 分区数或用户配置决定。

18. WordCount 生成物理查询计划

对 Spark WordCount：

```
sc.textFile("input")  
  .flatMap(_.split(" "))  
  .map((_, 1))  
  .reduceByKey(_ + _)  
  .saveAsTextFile("output")
```

逻辑计划是：

```
textFile -> flatMap -> map -> reduceByKey -> saveAsTextFile
```

物理计划要做两件事：**划分 Stage，划分 Task**。PPT里明确说，从逻辑到物理要做两件事情：划分 stage，划分 task。

可以写成：

Job: 由 saveAsTextFile 这个 Action 触发

Stage 0:

- 读取 HDFS 分区
- flatMap 拆词
- map 生成 (word, 1)
- 按 word 做 shuffle write

Stage 1:

- shuffle read
- reduceByKey 聚合
- saveAsTextFile 写 HDFS

Task 形成：

Stage 0 Task 数：由输入 block / partition 数决定

Stage 1 Task 数：由 shuffle partition 数决定

考试写法： Spark WordCount 中，flatMap 和 map 属于窄依赖，可以与读取输入放在同一个 Stage；reduceByKey 需要按 key 聚合，产生 Shuffle，因此在 reduceByKey 前后切分 Stage。第一个 Stage 完成读取、拆词、生成 `<word, 1>` 和 shuffle write；第二个 Stage 完成 shuffle read、按 key 求和，并通过 saveAsTextFile 写出结果。

19. RDD 的组成

PPT里说 RDD 包含四部分：**Partitions、Dependencies、Function、Metadata**。

Partitions，分块： RDD 的原子块。一个 RDD 分成多个 Partition，分布在不同节点上。

Dependencies，依赖： 描述当前 RDD 的 Partition 依赖父 RDD 的哪些 Partition。

Function，函数： 描述从父 RDD 到当前 RDD 要执行什么计算，比如 map 函数、filter 函数。

Metadata，元数据： 包括分区模式、数据位置等，用于调度和优化。

考试写法： RDD 由 Partition、Dependency、Function 和 Metadata 组成。Partition 是 RDD 的数据分块；Dependency 描述当前 RDD 与父 RDD 分区之间的依赖关系；Function 描述转换计算逻辑；Metadata 包含分区器和数据位置信息等。

20. 窄依赖和宽依赖

这是 Spark 原理最重要的判断题。

窄依赖 Narrow Dependency: 父 RDD 的每个 Partition 最多被一个子 RDD Partition 使用。常见操作：`map`、`filter`、`flatMap`。

图像理解：

```
父 P1 -> 子 P1
父 P2 -> 子 P2
父 P3 -> 子 P3
```

每条边不交叉，不需要 Shuffle。

宽依赖 Wide Dependency: 父 RDD 的一个 Partition 可能被多个子 RDD Partition 使用。常见操作：`groupByKey`、`reduceByKey`、默认未共分区的 `join`。

图像理解：

```
父 P1 -> 子 P1、子 P2、子 P3
父 P2 -> 子 P1、子 P2、子 P3
```

需要按 key 重分布数据，也就是 Shuffle。

PPT里定义：窄依赖是父 RDD 每个 partition 最多被一个子 partition 使用；宽依赖是父 partition 可能被多个子 partition 使用，并标注 narrow 与 shuffle 的判断。

考试写法：窄依赖指父 RDD 的每个 Partition 最多被一个子 RDD Partition 使用，通常不需要 Shuffle，可以在同一个 Stage 中流水线执行；宽依赖指父 RDD 的 Partition 可能被多个子 Partition 使用，需要进行 Shuffle，因此会导致 Stage 切分。

21. join 是窄依赖还是宽依赖？

这个要小心，不能绝对说 join 一定宽。

一般情况下，join 是宽依赖。 因为两个 RDD 要按 key 对齐，通常需要 Shuffle，把相同 key 的数据拉到同一个分区。

```
rdd1: (k, v1)
rdd2: (k, v2)
join 后要得到 (k, (v1, v2))
```

如果两个 RDD 原来没有按同一个 key、同一个 partitioner 分好区，就需要 Shuffle。

特殊情况下，**join** 可以是窄依赖。如果两个 RDD 已经使用相同 partitioner，并且相同 key 已经在对应分区中，那么 join 可以不重新 Shuffle，依赖可以看作窄依赖。

考试写法： join 通常是宽依赖，因为需要将两个 RDD 中相同 key 的记录重新分区到同一个节点进行匹配，会产生 Shuffle。但如果两个 RDD 已经按照相同的 partitioner 进行分区，并且分区对齐，则 join 可以避免 Shuffle，表现为窄依赖。

22. 分块和依赖的关系

Partition 是数据怎么切。**Dependency** 是切完以后，子 **Partition** 依赖哪些父 **Partition**。

比如 **map**：

```
父 RDD P1 -> 子 RDD P1'  
父 RDD P2 -> 子 RDD P2'
```

这是窄依赖，因为一个父分区只被一个子分区使用。

比如 **reduceByKey**：

```
父 RDD P1 里的不同 key 可能去不同子分区  
父 RDD P2 里的不同 key 也可能去不同子分区
```

这是宽依赖，因为父分区的数据要被打散到多个子分区。

考试写法： RDD 的分块决定数据被拆成哪些 Partition，依赖关系描述子 RDD 的每个 Partition 如何由父 RDD 的 Partition 计算得到。窄依赖中父分区和子分区关系较简单，可以流水线执行；宽依赖中父分区数据会被多个子分区使用，需要 Shuffle。

23. Spark 容错：lineage 重算

Spark 不像 HDFS 那样给每个中间结果都存多个副本。它主要靠 **lineage 血统** 容错。

lineage 是什么？ RDD 记录自己从哪些父 RDD、经过哪些算子计算得到。比如：

```
RDD1 = textFile  
RDD2 = RDD1.flatMap  
RDD3 = RDD2.map  
RDD4 = RDD3.reduceByKey
```

如果 RDD3 的某个 Partition 丢了，Spark 可以从 RDD1 对应的 Partition 开始，重新执行 flatMap 和 map，把它算回来。

窄依赖下容错更便宜： 只需要重算丢失的那条分区链。

宽依赖下容错更贵： 因为需要 Shuffle，某些中间 shuffle 文件丢了，可能要重算多个父分区。

考试写法： Spark 通过 RDD lineage 实现容错。RDD 会记录自己由哪些父 RDD 经过哪些 Transformation 生成，当某个 Partition 丢失时，Spark 可以根据 lineage 重新计算该 Partition。窄依赖下只需重算丢失分区对应的父分区链；宽依赖涉及 Shuffle，恢复成本可能更高。

24. “重新算一遍出错的”怎么理解

考试如果问“Spark 某个分区出错怎么办”，不要写“整张表重算”。正确写法是：

哪个 Partition 丢了，就根据 lineage 重算哪个 Partition。

例子：

HDFS block -> RDD1 P1 -> map -> RDD2 P1 -> filter -> RDD3 P1

如果 RDD3 P1 丢了，只需要从 RDD1 P1 开始重新执行 map 和 filter，算回 RDD3 P1。

但如果是宽依赖：

reduceByKey / groupByKey

可能需要重新生成 shuffle 输出，所以代价更高。

考试写法： Spark 的容错不是简单地把所有数据重新计算一遍，而是利用 lineage 定位丢失的 Partition，并重新执行生成该 Partition 所需的上游计算。对于窄依赖，只需要重算相关分区；对于宽依赖，可能涉及 Shuffle 数据恢复，重算范围更大。

本章高频简答模板

1. Spark 为什么比 MapReduce 快？ Spark 通过内存计算、RDD Cache、DAG 执行和线程级 Task 减少了 MapReduce 中频繁写磁盘、多轮作业通过 HDFS 交换数据和进程级任务启动的开销。对于迭代计算，Spark 可以缓存反复使用的数据，因此比 MapReduce 更高效。

2. RDD 是什么？ RDD 是弹性分布式数据集，是 Spark 的核心抽象。它是分布在集群中的只读对象集合，由多个 Partition 构成，可以存储在内存或磁盘中，通过 Transformation 构造，并通过 lineage 在分区丢失时自动重算。

3. Transformation 和 Action 的区别？ Transformation 输入 RDD 并输出新的 RDD，只记录转换关系，不立即执行；Action 输入 RDD，返回本地结果或写出外部系统，会触发 Spark 作业执行。判断方法是看结果是否还是 RDD。

4. Spark 的惰性执行是什么？ Spark 中 Transformation 不会立即执行，只会记录 RDD 的依赖关系。直到遇到 Action，Spark 才根据完整的 lineage 构建 DAG，划分 Stage 和 Task，并提交到集群执行。

5. Stage 怎么切分? Spark 根据依赖关系切分 Stage。窄依赖操作可以放在同一个 Stage 中流水线执行;遇到宽依赖或 Shuffle 操作时,需要切分 Stage。

6. Task 怎么形成? 每个 Stage 会按照 Partition 生成多个 Task,通常一个 Partition 对应一个 Task。第一阶段 Task 数通常由 HDFS block 或 HBase region 决定,后续 Stage Task 数通常由 Shuffle 分区数或用户配置决定。

7. 窄依赖和宽依赖区别? 窄依赖是父 RDD 的每个 Partition 最多被一个子 Partition 使用,不需要 Shuffle;宽依赖是父 Partition 可能被多个子 Partition 使用,需要 Shuffle。窄依赖可以流水线执行,宽依赖会导致 Stage 切分。

8. Spark 如何容错? Spark 通过 RDD lineage 容错。RDD 记录从父 RDD 到当前 RDD 的转换关系,当某个 Partition 丢失时,可以根据 lineage 重新计算该 Partition。窄依赖恢复成本低,宽依赖可能需要重新生成 Shuffle 数据。

本章易混点

Spark 没有自己的存储。 Spark 是计算框架,通常读写 HDFS / HBase。

RDD 是只读的。 每次转换产生新的 RDD,不是在原 RDD 上原地修改。

Transformation 不执行, Action 才执行。 `map/filter/reduceByKey` 不触发执行, `count/collect/saveAsTextFile` 才触发。

reduceByKey 是 Transformation, 但通常会产生 Shuffle。 不要因为它名字里有 reduce 就当成 Action。

一个 Action 触发一个 Job。 多个 Action 通常触发多个 Job。

Stage 按 Shuffle 切, 不是按算子个数切。 多个窄依赖算子可以在同一个 Stage 里。

join 通常是宽依赖, 但不是绝对。 如果两个 RDD 已经按同一 partitioner 共分区, join 可以避免 Shuffle。

本章看到题目怎么写

看到“**Spark 为什么快**”: 写内存计算、Cache、DAG、少落 HDFS、线程级 Task。

看到“**RDD 是什么**”: 写分布式、只读、Partition、Transformation 构造、可缓存、lineage 容错。

看到“**判断 Transformation / Action**”: 看输出是否还是 RDD; 写出去了或返回本地值就是 Action。

看到“**Spark WordCount**”: 写 `textFile -> flatMap -> map -> reduceByKey -> saveAsTextFile`, 前面 Transformation, 最后 Action。

看到“**惰性执行**”: 写 Transformation 只记录, Action 触发, 生成 DAG 再执行。

看到“**Stage 怎么切**”: 写窄依赖合并, 宽依赖 / Shuffle 切 Stage。

看到“**Task 怎么来**”: 写一个 Partition 一个 Task, 第一阶段由 HDFS block / HBase region 决定, 后续由 shuffle partition 决定。

看到“**RDD 容错**”: 写 lineage, 丢哪个 Partition 重算哪个 Partition, 宽依赖恢复更贵。